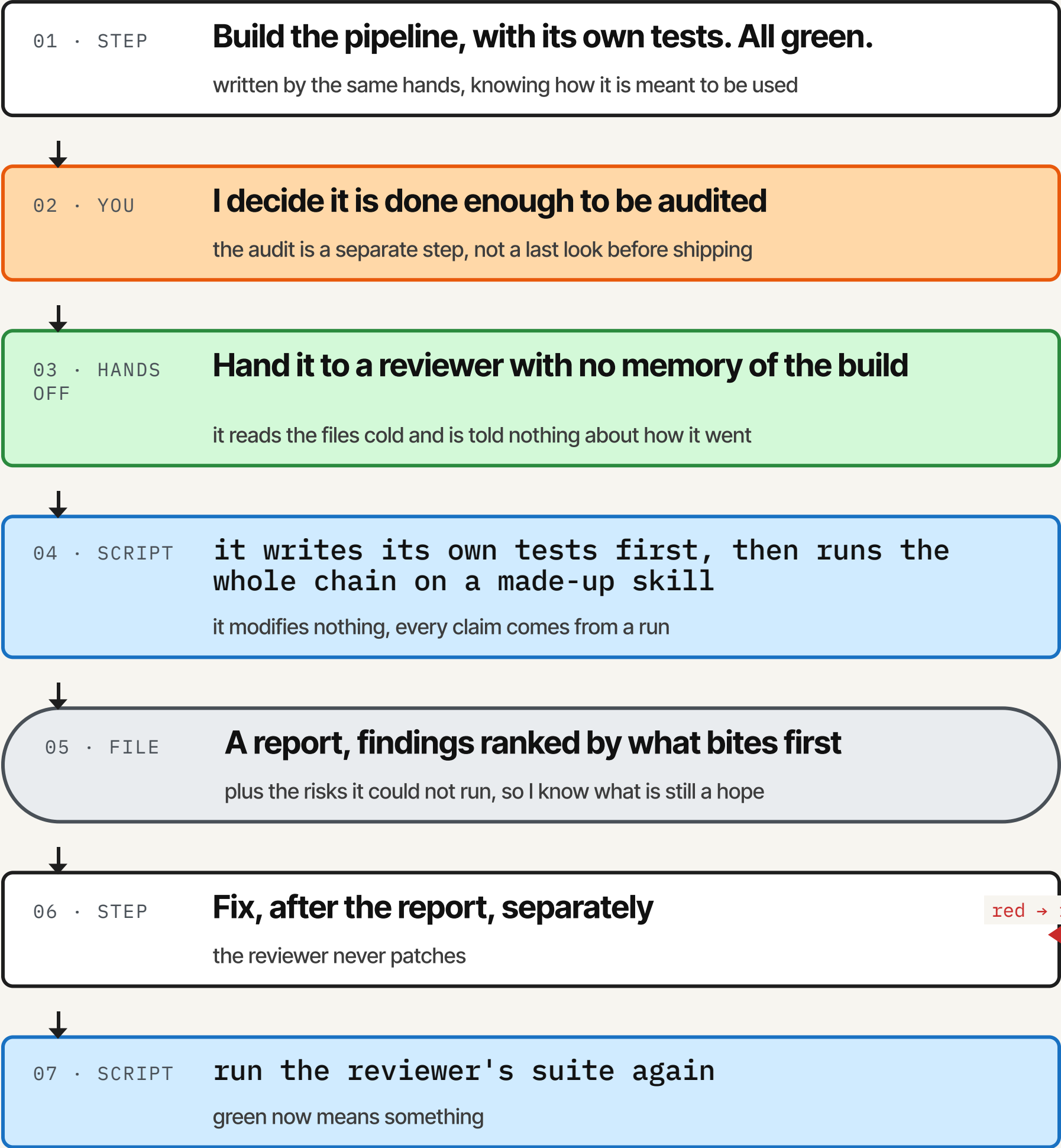


It passed its own tests. *Then it got audited.*

A pipeline that plans a skill before it is written. It had a test suite and everything was green. A second reviewer with no memory of the build wrote its own tests, ran the whole chain cold, and found the bug mine could not. A quiet success that deleted a decision and exited clean.

How it runs, *top to bottom*.



Whoever builds it does not audit it.

Eight failures *my own tests could not see.*

The reviewer's first run, then the same suite today.

FIRST RUN, WRITTEN COLD

INDEPENDENT AUDIT · FIRST RUN · WRITTEN COLD FROM THE FILES

Forty tests it wrote itself. Thirty-two pass. Eight fail.

THE EIGHT THAT FAILED

- ✗ a stale canvas silently deletes plan boxes added since the last render
- ✗ plan ids colliding with derived line ids force a false match in the diff
- ✗ delete every stage on the canvas, apply, re-render, crash
- ✗ the diff misses a dropped open question or a dropped why
- ✗ a one-line text edit the canvas does not re-wrap is missed
- ✗ shift-lines on a plan destroys it
- ✗ a plan with no name renders 'undefined' into the title
- ✗ a plan with a human box and no stages, crash

AFTER THE FIXES

THE SAME SUITE · AFTER THE FIXES · TODAY

Forty tests. Forty pass.

PIPELINE HAZARDS

- ✓ a stale canvas silently deletes plan boxes added since the last render
 - ✓ plan ids colliding with derived line ids must not force a false match
 - ✓ the diff notices when a plan's open question or why is dropped in the build
 - ✓ readback refuses deletions on a stale canvas, and writes a backup first
- suites 7 · tests 40 · pass 40 · fail 0

Green again. Which now means something, because the tests were not written by the person who built it.

The worst one exited clean. A stale canvas deleted plan boxes, kept no backup, and reported success.

The brief, *so you can run it on yours.*

Six lines. They work on any pipeline with files in it.

THE GIVE · THE AUDIT BRIEF, SO YOU CAN RUN IT ON YOURS

How to get a pipeline audited by something that did not build it

- A separate reviewer with no memory of the build. It reads the files cold.
- It writes its tests before it reads any of your answers.
- It runs the whole chain, end to end, on an invented input.
- It ranks what it finds by what would bite first, not by what is easiest to fix.
- It reports. It never fixes. Fixes happen after, separately, then the suite runs again.
- It says which risks it could not run, so you know what is still a hope.

Distilled from the brief I used. Whoever builds it does not audit it.

Green means more when someone else wrote the tests.